

1. Tutorial: construcción de un libro de cálculo con el DSL

Este capítulo narra, paso a paso, el proceso de construir un programa completo en el DSL a partir de la descripción de un problema de horario. El ejemplo es un **horario de defensas de tesis**: dado un conjunto de profesores, estudiantes y locales, se necesita un libro de cálculo que liste las defensas programadas, compute automáticamente cuántas defensas acumula cada profesor y resalte en rojo a los profesores con conflicto de horario.

La filosofía del sistema es **separar la descripción del problema de su generación**: el programador declara la estructura del libro (tablas, fórmulas, coloreado condicional) y el generador produce el archivo `.xlsx` de forma automática.

1.1 Requisitos

1.1.1 Software necesario

Para utilizar el generador se necesitan dos programas:

Steel Bank Common Lisp (SBCL) Intérprete de Common Lisp. Disponible en <https://www.sbcl.org/>. Versión mínima recomendada: 2.0.

Python 3.10 o superior Para ejecutar el código generado. Además de Python estándar, se requiere la biblioteca **openpyxl**:

```
pip install openpyxl
```

1.1.2 Estructura del proyecto

El código fuente del generador se organiza en los siguientes archivos:

```
generador-excel/  
+-- dsl-directo.lisp          <- DSL y macros (no modificar)  
+-- ast-def.lisp             <- definicion de nodos AST (no  
    modificar)  
+-- generate-code-direct.lisp <- backend de generacion (no  
    modificar)  
+-- code-generation-utils.lisp  
+-- hoja_con_formulas.py      <- backend Python (no modificar)  
\-- mi-horario/  
    +-- data-tutorial.lisp    <- datos concretos (el usuario  
        crea este)  
    \-- tutorial-horario.lisp <- programa DSL (el usuario crea  
        este)
```

Los archivos de la raíz son la **biblioteca del generador** y no deben modificarse. El usuario únicamente crea su propio programa DSL y su archivo de datos.

1.1.3 Ejecución

Hay dos formas equivalentes de ejecutar el flujo completo.

Opción A — script incluido (recomendada para comenzar):

```
cd tutorial/  
chmod +x generar.sh    # solo la primera vez  
./generar.sh
```

Opción B — carga directa con SBCL:

```
cd tutorial/  
sbcl --script tutorial-horario.lisp
```

Ambas opciones cargan el DSL, construyen el árbol AST, generan el programa de construcción del libro y lo ejecutan para crear el archivo `.xlsx`. La diferencia es que el script imprime mensajes de progreso y verifica que el archivo fue creado correctamente. La sección 1.8 explica el flujo con más detalle.

1.2 Conceptos fundamentales

1.2.1 El árbol de sintaxis abstracta (AST)

Cuando el usuario escribe definiciones en Common Lisp usando las macros del sistema, estas construyen automáticamente objetos en memoria que forman un **árbol de sintaxis abstracta (AST)**. Cada nodo del árbol representa un concepto del problema: una tabla, una expresión de celda, un cuantificador existencial, una regla de coloreado.

El usuario **nunca manipula el árbol directamente**: trabaja con las macros de alto nivel descritas en este documento.

1.2.2 El DSL es independiente del generador de código

El AST no contiene ninguna referencia a Excel, a Python ni a ninguna otra tecnología concreta. Es una representación **neutral del problema**. Cualquier componente que sepa recorrer el árbol e interpretar sus nodos puede actuar como generador de código para el destino que desee.

En este trabajo el generador implementado toma el AST y produce un **script Python** que usa la biblioteca `openpyxl` para construir el archivo `.xlsx`. Pero el mismo DSL podría alimentar, sin cambio alguno en el programa del usuario, un generador diferente que produjera, por ejemplo, una hoja LibreOffice, un informe PDF o una tabla HTML.

El generador implementado en este trabajo

El backend concreto que se distribuye con el DSL genera un script Python que usa `openpyxl` para construir archivos Excel (`.xlsx`). Las fórmulas de celda, los datos y las reglas de formato condicional quedan incrustados en el archivo y funcionan en cualquier versión de Excel o LibreOffice Calc sin necesidad de tener instalado Python ni SBCL.

1.2.3 Etapas de la canalización

El proceso de generación tiene tres etapas claramente separadas:

1. **Expansión de macros.** Cada forma del DSL (`def-table`, `exists`, `conditional-rendering`) se expande en un nodo del AST.
2. **Generación de código.** El árbol se pasa al generador, que recorre cada nodo y produce el programa de construcción del libro. En el backend incluido, ese programa es un script Python.
3. **Ejecución.** El programa generado se ejecuta y crea el archivo de salida con datos, fórmulas y reglas de formato condicional incrustadas.

1.2.4 Principio central: intención vs. implementación

El DSL declara *qué* debe ocurrir (“colorear los profesores con conflicto de horario”); el generador decide *cómo* implementarlo según el destino (para el backend Excel, genera una `FormulaRule SUMPRODUCT` que persiste y recalcula en la hoja). El programador del DSL no escribe ninguna fórmula ni ningún código del destino.

1.3 Del problema a las tablas

El primer paso es decidir cómo representar el dominio del problema en filas y columnas.

1.3.1 El problema

Se necesita planificar las defensas de tesis de una facultad. Cada defensa tiene un estudiante, un tribunal de cinco profesores (tutor, oponente, presidente, vocal y secretario), un día, una hora y un local. Además, se quiere saber cuántas defensas acumula cada profesor y detectar si algún profesor tiene dos defensas programadas a la vez.

1.3.2 Decidir cuántas tablas hacen falta

Decisión de diseño

La pregunta clave es: *¿qué es una fila?*

Si una fila es **una defensa**, la tabla tiene tantas filas como defensas y sus columnas son: estudiante, los cinco roles del tribunal, día, hora y local.

Si una fila es **un profesor**, la tabla tiene tantas filas como profesores y necesita una columna calculada que cuente en cuántas defensas participa.

El libro necesita *ambas* perspectivas: se usan dos tablas en la misma hoja que se referencian entre sí.

1.3.3 Tabla de defensas

La **tabla de defensas** (`defensas-table`) tiene una fila por defensa:

Columna	Descripción
<code>estudiante</code>	Nombre del estudiante que defiende
<code>tutor</code>	Profesor tutor
<code>oponente</code>	Profesor oponente
<code>presidente</code>	Presidente del tribunal
<code>vocal</code>	Vocal del tribunal
<code>secretario</code>	Secretario del tribunal
<code>dia</code>	Día de la defensa
<code>hora</code>	Hora de inicio
<code>local</code>	Nombre del local

Por qué los cinco roles van juntos y en ese orden

Los roles `tutor`, `oponente`, `presidente`, `vocal` y `secretario` son columnas consecutivas. Esta disposición permite referenciarlos como un rango continuo (`trange defensas-table tutor secretario`) al calcular cuántas veces aparece un profesor en cualquier rol, sin necesidad de sumar cinco búsquedas separadas.

1.3.4 Tabla de profesores

La **tabla de profesores** (`profesores-table`) tiene una fila por profesor:

Columna	Descripción
<code>nombre</code>	Nombre del profesor
<code>grado</code>	Grado académico
<code>defensas</code>	Número de defensas en que participa (calculada)

1.4 Definir la tabla de defensas

Sintaxis: def-table

```
1 (def-table nombre-tabla
2   ((col1 "Cabecera1") (col2 "Cabecera2") ...))
3   :cell-height N
4   :cell-width N)
```

Cada par (columna Cabecera") declara el nombre interno (símbolo DSL) y el texto del encabezado visual de la columna. Las opciones :cell-height y :cell-width controlan cuántas filas y columnas físicas de Excel ocupa cada fila o columna lógica (el valor 1 es el caso habitual).

Listing 1.1: Definición de defensas-table.

```
1 (def-table defensas-table
2   ((estudiante "") (tutor "") (oponente "") (presidente "")
3    (vocal "") (secretario "") (dia "") (hora "") (local ""))
4   :cell-height 1
5   :cell-width 1)
```

El valor "" en cada par indica que la columna se rellena con datos externos; no es un encabezado visible sino el valor por defecto de la celda antes de que el generador coloque las fórmulas.

1.5 Definir la tabla de profesores

La tabla de profesores introduce tres características nuevas: columnas calculadas, referencias cruzadas y formato condicional.

1.5.1 Columna calculada con :computed

La opción :computed asocia columnas a expresiones DSL que el generador traduce a fórmulas Excel. El valor de esa celda en los datos de entrada se ignora: la celda mostrará siempre el resultado de la fórmula.

Listing 1.2: Columna defensas calculada con COUNTIF.

```
1 (def-table profesores-table
2   ((nombre "") (grado "") (defensas ""))
3   :cell-height 1
4   :cell-width 1
5   :computed
6   ((defensas (countif (trange defensas-table tutor secretario
7                        (col nombre)))))
```

La expresión (countif rango criterio) genera la fórmula Excel COUNTIF(rango, criterio).

Cómo funciona trange

(trange defensas-table tutor secretario) genera un rango que abarca desde la columna tutor hasta la columna secretario de defensas-table, para todas las filas de datos. En Excel se traduce a algo como `$B$4:$F$6`.

La expresión completa cuenta cuántas veces aparece el nombre del profesor actual (tomado de su propia columna nombre con (col nombre)) en cualquiera de los cinco roles de cualquier defensa.

Se usa trange en lugar de range porque ambas tablas tienen una columna llamada nombre: trange especifica explícitamente la tabla de origen y evita la ambigüedad.

1.5.2 Formato condicional con :render

La opción :render acepta una forma conditional-rendering que especifica bajo qué condición se colorean determinadas columnas. La condición se expresa con el cuantificador exists.

Listing 1.3: Formato condicional sobre profesores-table.

```
1 (def-table profesores-table
2   ((nombre "") (grado "") (defensas ""))
3   :cell-height 1
4   :cell-width 1
5   :computed
6     ((defensas (countif (trange defensas-table tutor secretario
7                           )
8                           (col nombre))))
9   :render
10    (conditional-rendering
11     :condition
12       (exists (r :rows-of defensas-table)
13               :self-in (tutor oponente presidente vocal secretario
14                         )
15               :matching (dia hora))
16     :target-columns (nombre)))
```

La variable r es un alias documental de la fila que se comprueba, análogo a un alias de tabla en SQL.

Sintaxis: exists :rows-of

```
1 (exists (var :rows-of tabla-id)
2   :self-in (col1 col2 ...)
3   :matching (clave1 clave2 ...))
```



```

10      ("Luis Mora" "Lopez" "Torres" "Garcia" "Ruiz" "Soto"
11          "Martes" "10:00" "Aula
            1"))))

```

Las celdas con fórmulas calculadas se dejan vacías en el archivo de datos: el generador las sobrescribe con la fórmula correspondiente.

Listing 1.5: Datos de profesores-table.

```

1  (defparameter *DATOS-PROFESORES*
2    '((" " " " ")
3      (" " " " ")
4      ("Nombre" "Grado" "Defensas")
5      ("Piad" "Dr" " ") ;; "Defensas" se calcula con
6      COUNTIF
7      ("Garcia" "Dr" " ")
8      ("Lopez" "Msc" " ")
9      ("Torres" "Msc" " ")
10     ("Ruiz" "Lic" " ")
11     ("Soto" "Msc" " ")))

```

1.7 Construir el libro

Con las tablas declaradas y los datos preparados, se ensambla el workbook con tres macros: libro, hoja y tabla.

Sintaxis: libro / hoja / tabla

```

1  (libro nombre-workbook
2    :filename "nombre.xlsx"
3    :hojas (list
4      (hoja "Nombre hoja"
5        (tabla plantilla1 :data *DATOS-1*)
6        (tabla plantilla2 :data *DATOS-2*)
7        :fixed-expressions
8          (((nav ancla dir N) (expresion))
9            ...))))

```

La macro `tabla` instancia una plantilla (`def-table`) con datos concretos. Cuando una hoja recibe más de una `tabla`, el generador las coloca lado a lado dentro de la misma *región horizontal*, separadas por una columna en blanco.

La clave `:fixed-expressions` coloca expresiones o etiquetas fuera de las tablas. La posición se especifica con `nav`, que calcula la celda a partir de la última fila de una tabla más una serie de pasos direccionales.

Listing 1.6: Workbook completo con totales de pie de tabla.

```

1  (libro horario-tesis
2    :filename "Horario_Tesis.xlsx"
3    :hojas (list

```



```

4      (hoja "Defensas"
5        (tabla defensas-table :data *DATOS-DEFENSAS*)
6        (tabla profesores-table :data *DATOS-PROFESORES*)
7        :fixed-expressions
8          (((nav (ultima-fila defensas-table estudiante) abajo 1)
9              (str "Total defensas:"))
10           ((nav (ultima-fila defensas-table estudiante) abajo 1
11                 derecha 1)
12              (counta (trange defensas-table estudiante))))
13          ((nav (ultima-fila profesores-table nombre) abajo 1)
14              (str "Total profesores:"))
15          ((nav (ultima-fila profesores-table nombre) abajo 1
16                 derecha 2)
17              (sum-range (trange profesores-table defensas)))))))))

```

Sintaxis: nav / ultima-fila

```

1  ;; Posicion relativa a la ultima fila de una columna:
2  (nav (ultima-fila tabla-id columna) abajo N)
3  (nav (ultima-fila tabla-id columna) abajo N derecha M)
4  (nav (primera-fila tabla-id columna) arriba N)

```

Por qué usar nav en lugar de coordenadas fijas

Las coordenadas absolutas de la celda “una fila debajo de la última defensa” dependen de cuántas defensas haya en los datos. `nav` las calcula automáticamente en tiempo de generación, de modo que si se añaden más filas de datos los totales se desplazan solos sin modificar el programa DSL.

1.8 Generar y ejecutar

Las dos últimas líneas del programa desencadenan la generación del archivo:

Listing 1.7: Compilacion y ejecucion desde el programa DSL.

```

1  (xl-generate horario-tesis "horario-tesis.py")
2  (xl-run-generated "horario-tesis.py")

```

`xl-generate` recorre el AST del workbook y produce el programa de construcción del libro. `xl-run-generated` ejecuta ese programa con `python3`; el resultado es el archivo `Horario_Tesis.xlsx` en disco.

El archivo `.xlsx` contiene tanto las fórmulas de celda como las `FormulaRule` de formato condicional; estas últimas permanecen activas cuando el usuario abre y edita el libro directamente en Excel.

1.8.1 Ejecución desde la terminal con el script generar.sh

Como alternativa a invocar SBCL directamente, el directorio incluye un script Bash `generar.sh` que automatiza el mismo flujo:

```
# Primera vez: dar permisos de ejecucion
chmod +x generar.sh

# Ejecutar el flujo completo
./generar.sh
```

El script realiza exactamente los mismos tres pasos que ocurren al cargar el programa DSL a mano:

1. Sitúa el directorio de trabajo en la carpeta del tutorial (para que los `load` relativos del programa DSL funcionen independientemente del directorio desde el que se llame al script).
2. Invoca `sbcl --script tutorial-horario.lisp`, que expande las macros, construye el AST, genera `horario-tesis.py` y lo ejecuta con `python3`.
3. Verifica que `Horario_Tesis.xlsx` fue creado e imprime su tamaño.

¿Cuándo usar el script y cuándo cargar directamente?

El script es conveniente para ejecutar el tutorial de un solo paso desde cualquier directorio. Cargar el archivo directamente desde el REPL de SBCL (`(load "tutorial-horario.lisp")`) es preferible durante el desarrollo, porque permite inspeccionar el estado del AST en memoria entre las distintas formas del DSL.

1.9 El programa completo

A continuación se presenta el contenido íntegro de los dos archivos que conforman el programa.

1.9.1 *data-tutorial.lisp*

Listing 1.8: data-tutorial.lisp — datos de entrada.

```

1 (defparameter *DATOS-DEFENSAS*
2   '((( " " " " " " " " " " " " " ")
3     ( " " " " " " " " " " " " " ")
4     ("Estudiante" "Tutor" "Oponente" "Presidente" "Vocal"
5      "Secretario" "Dia" "Hora" "Local")
6     ("Juan Diaz" "Piad" "Garcia" "Lopez" "Torres" "Ruiz"
7      "Lunes" "10:00" "Aula 1")
8     ("Maria Vega" "Garcia" "Lopez" "Piad" "Soto" "Torres"
9      "Lunes" "14:00" "Aula 2")
10    ("Luis Mora" "Lopez" "Torres" "Garcia" "Ruiz" "Soto"
11     "Martes" "10:00" "Aula 1"))))
12

```

```

13 (defparameter *DATOS-PROFESORES*
14   '((" " " " ")
15     (" " " " ")
16     ("Nombre" "Grado" "Defensas")
17     ("Piad" "Dr" " ")
18     ("Garcia" "Dr" " ")
19     ("Lopez" "Msc" " ")
20     ("Torres" "Msc" " ")
21     ("Ruiz" "Lic" " ")
22     ("Soto" "Msc" " ")))

```

1.9.2 tutorial-horario.lisp

Listing 1.9: tutorial-horario.lisp — programa DSL completo.

```

1 (load "dsl-directo.lisp")
2 (load "data-tutorial.lisp")
3
4 ;; TABLA 1: defensas
5 ;; Nueve columnas: cinco roles consecutivos (tutor...secretario
6   ),
7 ;; dia, hora, local. Sin coloreado propio.
8 (def-table defensas-table
9   ((estudiante " ") (tutor " ") (oponente " ") (presidente " ")
10    (vocal " ") (secretario " ") (dia " ") (hora " ") (local " "))
11   :cell-height 1
12   :cell-width 1)
13
14 ;; TABLA 2: profesores
15 ;; Columna "defensas" calculada con COUNTIF.
16 ;; Coloreado condicional: nombre en rojo si el profesor tiene
17   conflicto.
18 (def-table profesores-table
19   ((nombre " ") (grado " ") (defensas " "))
20   :cell-height 1
21   :cell-width 1
22   :computed
23   ((defensas (countif (trange defensas-table tutor secretario
24   )
25   )
26   (col nombre))))
27
28 :render
29 (conditional-rendering
30   :condition
31   (exists (r :rows-of defensas-table)
32     :self-in (tutor oponente presidente vocal secretario
33   )
34     :matching (dia hora))
35   :target-columns (nombre)))
36
37 ;; WORKBOOK

```

```

32 (libro horario-tesis
33   :filename "Horario_Tesis.xlsx"
34   :hojas (list
35     (hoja "Defensas"
36       (tabla defensas-table :data *DATOS-DEFENSAS*)
37       (tabla profesores-table :data *DATOS-PROFESORES*)
38       :fixed-expressions
39       (((nav (ultima-fila defensas-table estudiante) abajo 1)
40         (str "Total defensas:"))
41        ((nav (ultima-fila defensas-table estudiante) abajo 1
42          derecha 1)
43         (counta (trange defensas-table estudiante)))
44        ((nav (ultima-fila profesores-table nombre) abajo 1)
45         (str "Total profesores:"))
46        ((nav (ultima-fila profesores-table nombre) abajo 1
47          derecha 2)
48         (sum-range (trange profesores-table defensas))))))
49 (xl-generate horario-tesis "horario-tesis.py")
50 (xl-run-generated "horario-tesis.py")

```

1.10 Resultado esperado

El programa genera Horario_Tesis.xlsx con la siguiente estructura:

Rango	Contenido
Columnas A–I, filas 4–6	defensas-table: tres defensas
Columna J	Separador en blanco
Columnas K–M, filas 4–9	profesores-table: seis profesores
Columna M, filas 4–9	Fórmulas COUNTIF generadas
Fila 7 (cols. A–B)	Totales de defensas (COUNTA)
Fila 11 (cols. K, M)	Totales de profesores (SUM)
Rango K4:K9	FormulaRule: nombres en rojo si hay conflicto

Con los datos de ejemplo, las defensas de “Juan Díaz” y “Luis Mora” comparten el slot **Lunes 10:00** con roles solapados, por lo que los profesores Piad, García, López, Torres, Ruiz y Soto aparecen resaltados en rojo.

1.11 Traza del diseño

El cuadro siguiente resume las decisiones tomadas durante el modelado, relacionando cada requisito del problema con su representación en el DSL.

Requisito	Representación	Decisión de diseño
Una fila por defensa con tribunal completo	<code>defensas-table</code>	Granularidad natural; agrupa todas las columnas de tribunal
Los cinco roles en columnas consecutivas	<code>orden</code> en <code>def-table</code>	Permite <code>trange tutor secretario</code> para contar roles
Contar defensas por profesor	<code>columna</code> <code>computed</code>	<code>countif</code> sobre el rango de roles de <code>defensas-table</code>
Detectar profesores con conflicto	<code>:render</code> con <code>exists</code> <code>:rows-of</code>	El backend genera una <code>FormulaRule</code> que persiste en Excel
Totales dinámicos al pie de tabla	<code>:fixed-expression</code> con <code>nav</code>	Las coordenadas se recalculan si cambia el número de filas

1.12 Referencia rápida de formas

Forma	Descripción
<code>(def-table nom cols opts...)</code>	Define una plantilla de tabla
<code>:computed ((col expr))</code>	Columna calculada con fórmula Excel
<code>:render (conditional-rendering)</code>	Regla de coloreado condicional
<code>(conditional-rendering :condition :target-columns)</code>	Asocia condición con columnas a colorear
<code>(exists (r :rows-of tabla) ...)</code>	Existencia en otra tabla (cross-table)
<code>(exists (r :all-rows) ...)</code>	Existencia en la misma tabla
<code>(col nombre)</code>	Valor de la columna en la fila actual
<code>(trange tabla desde hasta)</code>	Rango en otra tabla
<code>(countif rango criterio)</code>	Fórmula COUNTIF
<code>(counta rango)</code>	Fórmula COUNTA
<code>(sum-range rango)</code>	Fórmula SUM
<code>(str "texto")</code>	Literal de texto en celda
<code>(nav (ultima-fila t c) dir N)</code>	Posición relativa a última fila
<code>(nav (primera-fila t c) dir N)</code>	Posición relativa a primera fila
<code>(libro nom :filename :hojas)</code>	Crea el workbook
<code>(hoja nom tablas :fixed-expressions)</code>	Define una hoja del libro
<code>(tabla plantilla :data datos)</code>	Instancia una plantilla con datos
<code>(xl-generate wb archivo)</code>	Genera el programa de construcción
<code>(xl-run-generated archivo)</code>	Ejecuta el programa y crea el .xlsx

1.13 Formas del DSL no utilizadas en este tutorial

El tutorial cubre un subconjunto del DSL suficiente para construir el horario de defensas. Las formas que se describen a continuación están disponibles y resultan útiles en escenarios más complejos.

Condicionales en celdas

Permiten que el valor de una columna dependa de una condición evaluada fila a fila.

Forma	Para qué sirve
<code>(_if cond entonces sino)</code>	IF de tres ramas. Muestra un valor sólo cuando se cumple la condición; en caso contrario muestra otro.
<code>(non-empty expr)</code>	Verdad si la expresión no está vacía (<>). Útil como condición de <code>_if</code> .
<code>(show-nothing)</code>	Produce la cadena vacía <code>.</code> Se usa como rama <i>sino</i> para dejar la celda en blanco.
<code>(equals a b) / (different a b)</code>	Igualdad y desigualdad entre dos expresiones.
<code>(_and a b) / (_or a b)</code>	Conjunción y disyunción lógica.

Ejemplo: mostrar el nombre de una sala sólo si la fila tiene un evento asignado, y dejarla en blanco en caso contrario.

```
1 (col "Sala" :computed
2   (_if (non-empty (col "Evento"))
3     (col "Sala")
4     (show-nothing)))
```

Navegación entre filas

Permiten que una columna acceda al valor de la misma celda en la fila anterior o siguiente, lo que es indispensable para calcular diferencias temporales entre registros consecutivos.

Forma	Para qué sirve
<code>(previous-row expr)</code>	Desplaza la referencia una fila hacia arriba.
<code>(previous-of base)</code>	Valor de <code>base</code> en la fila inmediatamente anterior.
<code>(next-of base)</code>	Valor de <code>base</code> en la fila siguiente.
<code>(it-is-the-first-row)</code>	Verdad en la primera fila de datos. Se combina con <code>_if</code> para el caso borde donde no existe fila anterior.
<code>primera-fila</code>	Ancla de posición a la primera fila de la tabla; complemento de <code>ultima-fila</code> .

Ejemplo: calcular la duración de cada franja horaria como la diferencia entre su hora de fin y la hora de fin de la fila anterior.

```

1 (col "Duracion" :computed
2   (_if (it-is-the-first-row)
3     (str "")
4     (subtract (col "Hora fin")
5               (previous-of (col "Hora fin")))))

```

Aritmética y texto

Forma	Para qué sirve
(add a b) / (subtract a b)	Suma y resta sobre valores numéricos o de celda.
(multiply a b) / (divide a b)	Multiplicación y división.
(time-add hora minutos)	Suma una cantidad de minutos a una hora Excel (hora + minutos/1440).
(concat a b)	Concatenación de texto: combina el contenido de dos expresiones en una sola cadena.

Ejemplo: calcular la hora de fin a partir de la hora de inicio y una duración fija en minutos.

```

1 (col "Hora fin" :computed
2   (time-add (col "Hora inicio") (col "Duracion min")))

```

Referencias avanzadas entre tablas

Forma	Para qué sirve
(tcol tabla nombre)	Como col, pero especifica explícitamente de qué tabla proviene la columna. Necesario cuando dos tablas de la misma hoja comparten un nombre de columna y hay que desambiguar.
(range desde hasta)	Rango de columnas de la tabla <i>actual</i> , sin necesidad de nombrarla. Equivalente de <code>trange</code> para la tabla en curso.
(lookup clave campo)	Búsqueda estilo VLOOKUP: dado el valor de clave en la fila actual, devuelve el valor de campo en la tabla de referencia.

Plantillas paramétricas

Cuando una misma estructura de tabla debe instanciarse varias veces con un detalle variable —por ejemplo, la misma tabla de franjas para cada día de la semana— se declaran *inst-params* en `def-table`:

```

1 (def-table franja-dia
2   ((turno "") (salon "") (grupo ""))

```



```

3  :inst-params (dia)
4  :computed
5  ((salon (tcol sala-disponible salon)))

```

Dentro de la definición, (param nombre) accede al valor que se pase al instanciar. Al construir la hoja se indica el parámetro como argumento con nombre:

```

1  (hoja "Semana"
2    (tabla franja-dia :dia 'lunes :data *datos-lunes*)
3    (tabla franja-dia :dia 'martes :data *datos-martes*))

```

Con este mecanismo el DSL genera cinco hojas con estructura idéntica y fórmulas parametrizadas con el nombre del día, a partir de una sola declaración `def-table`.

Detección de solapamiento dentro de la misma tabla

El tutorial usa (exists (r :rows-of tabla) ...) para buscar conflictos en una tabla *distinta*. La variante `:all-rows` detecta solapamientos dentro de la *propia* tabla:

```

1  (conditional-rendering
2    (exists (r :all-rows)
3      :matching (:col-a "Inicio" :col-b "Fin")
4      :any-overlap t)
5    :format '(:bg "#FF9999"))

```

Útil, por ejemplo, para marcar franjas que coinciden en la misma tabla de turnos de un único recurso.

Celdas de varias filas o columnas

Las opciones `:cell-height N` y `:cell-width N` de `def-table` hacen que cada fila lógica ocupe N filas (o columnas) físicas en el Excel generado. Es adecuado para horarios donde cada franja horaria debe abarcar varias celdas de la cuadrícula, o para encabezados que fusionan columnas.

```

1  (def-table horario-visual
2    ((hora "") (actividad "") (sala ""))
3    :cell-height 2) ; cada franja ocupa 2 filas físicas

```

Hojas con tablas apiladas verticalmente

La macro `hoja-v` es la versión vertical de `hoja`: apila las tablas una encima de otra en lugar de colocarlas en columnas. Se usa cuando las tablas tienen muchas columnas y el libro se orienta para impresión en apaisado, o cuando las tablas representan secciones consecutivas de un mismo listado.

```

1  (hoja-v "Resumen"
2    (tabla tabla-manana :data *datos-manana*)
3    (tabla tabla-tarde :data *datos-tarde*))

```

Referencias cruzadas entre hojas

Para libros con varias hojas que necesitan compartir datos, el DSL ofrece dos formas:

Forma	Para qué sirve
<code>(sheet-ref hoja plantilla-celda)</code>	Genera una referencia a una celda de otra hoja, expresada como plantilla de texto en la que <code>\${row-num}</code> se sustituye por el número de fila real.
<code>(cross-cell :sheet var :col col :row fila)</code>	Referencia a una celda concreta de otra hoja, identificada por su variable de hoja, columna y fila. Se usa cuando es necesario leer un campo de cada hoja en un bucle de generación.